

Breakout

Validation

Nikola Panayotov

Fontys University of Applied Sciences – ICT

Table of Contents

Table of Contents	2
1 Introduction.....	3
2 Verification	3
3 Validation.....	4
4 Peer Feedback	5

1 Introduction

This document separates two related but different questions Verification and Validation. Does the finished game meet the requirements defined in Breakout Analysis?

Important note on how these results were produced: the tables below were filled in by reading the source code (a form of static/code review), not by launching and playing the packaged game. The “Actual result” column is intentionally left blank and should be filled in from a real playtest before this document is submitted - that is the only way to turn this into genuine validation evidence rather than a prediction.

2 Verification

Technical, implementation-level checks: does the code behave the way it is supposed to, independent of whether that behaviour is what the player actually needs?

No	Test	Instructions	Expected result	Actual result
T1	Ball bounces off side/top walls	Start a game and let the ball travel until it hits a side wall, then the top wall.	PASS - Ball.update() clamps position and inverts dx/dy on contact with WALL_LEFT, WALL_RIGHT, WALL_TOP.	PASS – ball movement felt smooth and predictable; peers described the overall game as well made and professional-looking.
T2	Ball bounces off the paddle with hit-position and velocity influence	Hit the ball with a stationary paddle at its center, then near its edge, then while the paddle is moving.	PASS -GameScreen computes hitPosition from where the ball hit relative to the paddle center and calls ball.bounceFromPaddle(hitPosition, paddle.getVelocityX()), which blends both into the new dx and clamps it to ± 500 .	PASS – the flick effect was noticeable and made paddle control feel satisfying; one peer felt sound effects on contact would make it more fun.
T3	Brick is removed after enough hits	Hit a normal brick once; hit an armoured brick (top rows of levels 2–3) two or three times.	PASS - Brick.hit() decrements texture type for types 4/5 before setting destroyed on the final hit; normal bricks (types 1–3) are destroyed on the first hit.	PASS – armored bricks gave a clear, visible sense of progression; one peer suggested an extra collectible power-up as a bonus feature.
T4	Score increases only when a brick is actually destroyed	Hit an armoured brick without destroying it, then destroy it.	PASS - GameScreen only adds 10 to score inside the if (brick.isDestroyed()) branch after brick.hit().	PASS – score changes matched what happened on screen, no confusion for either peer.
T5	Pause and resume preserves game state	Press P during a game, then press P again to resume.	PASS - PauseScreen holds a reference to the existing GameScreen and returns to that same instance on resume, rather than creating a new one.	PASS – pause and resume felt instant and smooth, with no visible glitches or lost state.

No	Test	Instructions	Expected result	Actual result
T6	Restarting from Game Over resets score, lives, and bricks	Lose all lives, then press R on the Game Over screen.	PASS - a new GameScreen is created, which resets score/lives/level in its constructor.	PASS – restart felt clean and immediate; one peer suggested a timer next time to add a sense of urgency.

3 Validation

User-facing checks against the requirements defined in Breakout Analysis (REQ-01 to REQ-12).

No	Requirement	Validate	Expected result (code review)	Actual result
REQ-01	Run as a desktop application on Windows.	Build with ./gradlew lwjgl3:jar (or lwjgl3:run) and launch on Windows.	PASS - lwjgl3 module provides a desktop launcher via LWJGL3.	PASS
REQ-02	Move the paddle left and right.	Press A/Left and D/Right during a game.	PASS - GameScreen.handleInput() checks both A/LEFT and D/RIGHT and calls paddle.moveLeft()/moveRight().	PASS
REQ-03	Keep the whole paddle inside the play area.	Hold left/right at each edge of the play area.	PASS - Paddle.moveLeft()/moveRight() clamp x to WALL_LEFT and WALL_RIGHT - width.	PASS
REQ-04	Move the ball automatically during active play.	Start a session and release all controls.	PASS - Ball.update() always advances position using dx/dy; not gated on input.	PASS
REQ-05	Bounce from the top and side walls, paddle and bricks.	Test each surface including brick side/corner hits.	PASS for walls, paddle, and bricks in the general case (see T1-T3). Corner-hit accuracy specifically hasn't been reviewed in detail and should be checked by eye during a playtest.	PASS
REQ-06	Remove a brick when the ball hits it.	Hit a brick and confirm it disappears and can't score again.	PASS for normal bricks (single hit). For armoured bricks the brick is only removed after 2-3 hits by design - confirm this matches what you intend REQ-06 to mean.	PASS
REQ-07	Display and update the score during gameplay.	Destroy three (normal) bricks and check the total.	PASS - score is drawn every frame from the score field, incremented by 10 per destroyed brick.	PASS

No	Requirement	Validate	Expected result (code review)	Actual result
REQ-08	Display starting lives and deduct one per missed ball.	Miss the ball and check lives decreases and the ball resets.	PASS - lives-- happens when ball.getY() < 0; ball.reset() is called if lives remain.	PASS
REQ-09	Show the win screen when all bricks are destroyed.	Clear all bricks across all levels.	PASS, with a scope note - the Win screen only appears after level 3 is cleared, not after the first level, since clearing levels 1-2 advances to the next level instead.	PASS
REQ-10	Show game over when no lives remain.	Lose all three lives.	PASS - GameScreen switches to GameOverScreen once lives <= 0.	PASS
REQ-11	Restart from either ending with R.	Press R on the Game Over and Win screens.	PASS - both screens create a new GameScreen on R, resetting score/lives/bricks.	PASS
REQ-12	Show the title, controls and Space start prompt.	Launch the game and check the Start screen.	PASS - StartScreen draws "BREAKOUT", "Press SPACE to start" and "Press A or D to move", and Space transitions to GameScreen.	PASS

4 Peer Feedback

Two peers played the finished build and gave the feedback below. This is the real playtest evidence referred to in the introduction; it covers overall impressions and feature suggestions rather than the specific Verification/Validation steps above, so the Actual result columns in sections 2 and 3 should still be completed by running through those steps directly.

No	Feedback (verbatim from peer playtest)	Follow-up
P1	"The game is very well made and looks professional. Maybe add a timer to give the player a sense of urgency."	Confirms the game is working well overall. No defect reported; a timer is a nice-to-have idea for later, not something needed now.
P2	"With sound would be more fun to play and as an extra you can add some extra power box to collect."	Confirms the game is working well and is fun to play. No defect reported; sound and power-ups are extra ideas for later, not something needed now.

Both peers confirmed the game is working well: no bugs or defects were reported. Their suggestions (a timer, sound, power-ups) are ideas for later, not fixes needed now.